

login.js — Fundamentos

Explicación detallada de la pantalla de inicio de sesión (GymBro App)

1. El panorama general: cómo encaja login.js en la app

Archivo raíz: `app/index.tsx`

```
<AuthProvider>          <- guarda "quien esta logueado" (estado global)
  <NavigationContainer>
    <AppNavigator />      <- decide que pantallas existen segun haya o no usuario
AppNavigator (líneas 21-52) lee const { user } = useAuth() y arma un Stack.Navigator
condicional:
```

- Si `!user` → solo existen las rutas login y signup.
- Si `user` y `user.isTrainer` → rutas de entrenador.
- Si `user` y no es entrenador → rutas de usuario común (`home_user`, etc.).

Punto clave para defender: esto no es “login que navega a home”. Es que el propio árbol de navegación cambia cuando `user` deja de ser null. Por eso en `login.js` nunca vas a ver un `navigation.navigate('home_user')` después de loguear — no existe esa ruta en el stack cuando no hay usuario, y cuando el usuario se loguea, React vuelve a renderizar `AppNavigator`, que ahora arma otro stack con otras pantallas. React Navigation reemplaza el navegador entero y muestra la primera screen del nuevo grupo (`home_user` o las de entrenador). El comentario del código lo dice explícito: “Si son correctas, el navegador cambia de pantalla solo.”

2. AuthContext.js — el “backend falso”

Es un React Context que centraliza el estado de sesión (`user`) y las tres operaciones: login, register, logout.

```
const login = (email, password) => {
  const found = users.find(u =>
    u.email.toLowerCase() === email.trim().toLowerCase() &&
    u.password === password
  );
  if (!found) return { ok: false, error: 'Email o contraseña incorrectos' };
  setUser(found);
  return { ok: true };
};
```

- Busca en el array **hardcodeado** `users.js` (no hay backend real, ni hash de contraseñas, ni tokens — todo en memoria del proceso de JS).
- Compara email normalizado (trim + lowercase, para que 'Sol@Gmail.com' y 'sol@gmail.com' sean iguales) y password tal cual.
- Si encuentra match, hace `setUser(found)` → esto dispara el re-render de `AppNavigator` mencionado arriba.
- Devuelve un objeto `{ ok, error }` en vez de lanzar excepciones o usar promesas: es un patrón “Result object” simple, sincrónico, ideal para explicar que no hay llamadas de red (todo es local/simulado).

Si un profesor pregunta “¿dónde persiste la sesión?”: **no persiste**. user vive en un useState de React. Si recargás la app, se pierde y vuelve a login. No hay AsyncStorage/SecureStore ni JWT. Es un punto débil real que podés reconocer con seguridad (“es un mock de autenticación, pensado para la demo; en producción usaríamos un backend con hashing y persistencia de sesión”).

3. login.js paso a paso

Estado local (líneas 20-23)

```
const [email, setEmail] = useState('');
const [password, setPassword] = useState('');
const [error, setError] = useState('');
```

Son **inputs controlados**: React es la única fuente de verdad del valor de los campos (no el DOM/TextInput nativo). Por eso CustomInput recibe value y onChangeText.

handleLogin (líneas 27-35)

```
const handleLogin = () => {
  if (!email.trim() || !password) {
    setError('Completa email y contraseña');
    return;
  }
  const result = login(email, password);
  if (!result.ok) setError(result.error ?? '');
};
```

Dos capas de validación:

- **Validación de UI** (cliente): campos no vacíos. email.trim() evita que “ ” cuente como válido. Nota: password no se hace trim (a propósito, una contraseña con espacios podría ser válida).
- **Validación de negocio**: delega en login() del contexto. Si falla, muestra el mensaje de error que devuelve el contexto (no un mensaje propio de la pantalla), así el mensaje “Email o contraseña incorrectos” vive en un solo lugar (AuthContext).

Si el login es exitoso, **no hay ningún** if (result.ok) navigation.navigate(...) — justamente porque no hace falta: el cambio de pantalla es un efecto colateral de que AppNavigator se re-renderiza al cambiar user.

Por qué setError("") en cada onChangeText (líneas 69-72, 81-84)

Cada vez que el usuario toca el email o la contraseña, se borra el error visible. Es UX estándar: el mensaje de error no debe quedar pegado en pantalla mientras el usuario ya está corrigiendo lo que escribió.

Renderizado condicional del error (línea 97)

```
{error !== '' && <Text style={styles.error}>{error}</Text>}
```

Patrón típico de React: && como “si-entonces” para JSX. Si error es "" (string vacío, falsy), no se renderiza nada.

Navegación a signup (línea 108)

```
<TouchableOpacity onPress={() => navigation.navigate('signup')}>
```

Esta sí es una navegación explícita, porque signup **sí** existe en el stack actual (grupo “sin sesión”). navigation llega como **prop** porque LoginScreen está registrada como component={LoginScreen} dentro de un Stack.Screen — React Navigation inyecta automáticamente navigation y route como props a cualquier componente usado así.

4. Piezas reutilizables que usa

- **CustomInput** (src/components/CustomInput.js): input controlado genérico con label y, si `secureTextEntry` es true, un ícono de ojo (Ionicons) que alterna un estado local visible para mostrar/ocultar la contraseña. Nota importante: `secureTextEntry={secureTextEntry && !visible}` — el ocultamiento real lo hace la prop nativa de `TextInput`, el ojito solo la togglea.
- **PrimaryButton**: `TouchableOpacity` estilizado, sin lógica, recibe `title` y `onPress`.
- **useResponsiveLayout**: usa `useWindowDimensions` de RN para saber si el layout está en horizontal (`isLandscape`) y aplica estilos extra (`scrollContentLandscape`, `cardLandscape`) — nada relacionado a la lógica de auth, es puramente estético/responsive.
- **SafeAreaView + ScrollView**: `SafeAreaView` evita que el contenido quede debajo del notch/status bar; el `ScrollView` con `keyboardShouldPersistTaps="handled"` permite tocar botones aunque el teclado esté abierto sin que se cierre antes de registrar el tap.

5. Preguntas típicas que te pueden hacer y cómo responderlas

- **¿Qué pasa si dos usuarios tienen el mismo email?** → No se valida al loguear (usa `.find`, toma el primero), pero register en `AuthContext.js` sí valida que no exista duplicado de email ni DNI al crear cuenta.
- **¿Las contraseñas están hasheadas?** → No, están en texto plano en `users.js`. Es una limitación reconocida del mock, no de un sistema real.
- **¿Cómo sabe la app qué pantalla mostrar tras loguear?** → Por el user en contexto que reconstruye el `Stack.Navigator`.
- **¿Por qué separaron login.js de AuthContext.js?** → Separación de responsabilidades: la pantalla solo maneja UI/estado de formulario; la lógica de “es válida esta credencial” vive en el contexto para poder reutilizarla (y para poder cambiarla por un backend real sin tocar la pantalla).
- **¿Qué es useAuth?** → Un hook de conveniencia (`useContext(AuthContext)`) para no repetir `useContext(AuthContext)` en cada pantalla.